

Exposé zur Bachelorarbeit

Erweiterung des Tensor Power Flow um PV-Knoten:
Methoden, Implementierung und Vergleich mit
konventionellen Lastflussverfahren

25. Juni 2026

Studierender: Sebastian Wenzel Grigorevski

Erstbetreuer: Mike Vogt

Zweitbetreuer: who knows

Fraunhofer IEE

Universität Kassel

Geplanter Beginn: 01.06.2026

Bearbeitungsdauer: 10 Wochen

1 Problemstellung und Motivation

Die Lastflussberechnung (Power Flow, PF) ist eine der fundamentalsten Analysen in der elektrischen Energietechnik. Moderne Anwendungen wie Zeitreihensimulationen (TSS), probabilistische Lastflüsse (PPF), metaheuristik-basierte Optimierungen und maschinelles Lernen erfordern die Berechnung von Hunderttausenden bis Millionen einzelner Lastflüsse [1]. Bei einer Jahressimulation mit 1-Minuten-Auflösung sind beispielsweise $\tau = 525.600$ Lastflussberechnungen notwendig.

Der kürzlich vorgestellte **Tensor Power Flow (TPF)** [1] adressiert diese Herausforderung durch eine auf der Fixed-Point-Iteration (FPI) basierende Formulierung, die durch Tensorisierung (Reshape zu Matrix-Matrix-Operationen) massiv parallelisierbar ist. Die zentrale Iterationsvorschrift lautet:

$$v_{(n+1)} = Z_B \cdot \left(s^* \odot v_{(n)}^{*\odot(-1)} \right) + w \quad (1)$$

wobei $Z_B = Y_{dd}^{-1}$ die Bus-Impedanzmatrix, s der Vektor der komplexen Knotenleistungen, $v_{(n)}$ der Spannungsvektor in Iteration n , und $w = -Z_B Y_{ds} v_s$ ein Konstantvektor ist. Im Vergleich zum Newton-Raphson-Verfahren in seiner Sparse-Formulierung wurde ein Speedup-Faktor von bis zu **164** nachgewiesen [1]. Dieser Geschwindigkeitsvorteil resultiert daraus, dass die Matrix Z_B über alle Iterationen und alle τ gleichzeitig gelösten Lastflüsse **konstant** bleibt, was die Parallelisierung auf CPUs und GPUs erst ermöglicht.

Allerdings weist der TPF eine wesentliche **Limitation** auf: Er unterstützt ausschließlich **Slack- und PQ-Knoten**. PV-Knoten (spannungsgeregelte Knoten mit vorgegebener Wirkleistung P und Spannungsbetrag $|V|$) können nicht direkt abgebildet werden, da deren unbekannte Blindleistung Q den Leistungsvektor s variabel macht und somit die Konstanz der Systemmatrix verletzt. Mit zunehmender Integration von Synchrongeneratoren und großen PV-Anlagen mit $Q(U)$ -Regelung in Verteilnetzen gewinnt die korrekte Abbildung spannungsregelnder Einspeiser jedoch an Bedeutung.

2 Zielsetzung und Forschungsfragen

2.1 Ziel der Arbeit

Entwicklung, Implementierung und vergleichende Bewertung von Methoden zur Integration von PV-Knoten in den Tensor Power Flow, wobei der Parallelisierungsvorteil des TPF möglichst erhalten bleiben soll.

2.2 Forschungsfragen

Tabelle 1: Forschungsfragen der Arbeit

Nr.	Forschungsfrage
F1	Welche Methoden zur Q-Korrektur sind geeignet, PV-Knoten im FPI-basierten TPF abzubilden, ohne die Parallelisierbarkeit wesentlich einzuschränken?
F2	Bleibt der Geschwindigkeitsvorteil des TPF gegenüber konventionellen Verfahren (NR, BFS) bei Netzen mit PV-Knoten signifikant erhalten ?
F3	Wie verhält sich die Konvergenz der vorgeschlagenen Methoden in Abhängigkeit der Anzahl der PV-Knoten (n_{PV}/b) und des Lastfaktors λ ?
F4	Welche der untersuchten Methoden bietet das beste Verhältnis aus Genauigkeit, Rechenzeit und Robustheit?

3 Methodik

3.1 Theoretische Grundlagen

Die Arbeit baut auf dem FPI-Algorithmus nach Giraldo et al. [2] und dessen Tensorisierung nach Salazar Duque et al. [1] auf. Der Basisalgorithmus (Successive Approximation Method, SAM) lautet gemäß [2], Gl. (15):

$$V_d^{(k+1)} = B^{-1} \left(A^{(k)} V_d^{*(k)} - C - D^{(k)} \right) \quad (2)$$

wobei $B = \text{diag}(\alpha_Z \odot s^*) + Y_{dd}$ (konstant) und $A^{(k)} = \text{diag}(\alpha_P \odot \mathbf{V}_0^{*-2} \odot s_n^*)$ (iterationsabhängig) sind.

Die zentrale Herausforderung besteht darin, dass bei PV-Knoten die Blindleistung Q_k unbekannt ist und der Leistungsvektor $s_k = P_k^{\text{spec}} + jQ_k$ somit iterativ bestimmt werden muss, was die Grundannahme des TPF ($s = \text{konstant}$) verletzt.

3.2 Zu untersuchende Methoden

3.2.1 Methode A: Äußere Q-Schleife

Der Standard-TPF wird als innere Schleife (mit festem Q) ausgeführt. Eine äußere Schleife aktualisiert Q basierend auf der Sensitivität der Spannungsbedingung:

$$Q_k^{(\ell+1)} = Q_k^{(\ell)} + \frac{|V_k^{\text{spec}}|^2 - |v_k^{(\ell)}|^2}{2 \cdot X_{kk}} \quad (3)$$

wobei $X_{kk} = \text{Im}(Z_{B,kk})$ die Thévenin-Reaktanz am Knoten k ist und $v_k^{(\ell)}$ die konvergierte Spannung der inneren Schleife nach der ℓ -ten äußeren Iteration.

3.2.2 Methode B: Eingebettete Q-Korrektur (Interleaved Update)

Im Gegensatz zu Methode A, die eine verschachtelte Schleifenstruktur erfordert, integriert Methode B die PV-Knoten-Behandlung direkt in jede FPI-Iteration. Die Grundidee besteht darin, nach dem FPI-Schritt die Spannung am PV-Knoten auf den vorgegebenen Betrag zu projizieren und anschließend die Blindleistung exakt aus der Leistungsbilanz zurückzurechnen.

Der modifizierte Algorithmus. In jeder Iteration n werden die folgenden vier Schritte ausgeführt:

Schritt 1: Standard-FPI mit aktuellem Leistungsvektor $s^{(n)}$:

$$v_{(n+1)} = \cdot \left(s^{(n)*} v_{(n)}^* \right) + w \quad (4)$$

Schritt 2: Spannungsbetrags-Projektion an den PV-Knoten. Der Spannungszeiger wird in seiner Richtung (Winkel) beibehalten, aber im Betrag auf den Sollwert skaliert:

$$v_k^{(n+1)} \leftarrow |V_k| \cdot \frac{v_k^{(n+1)}}{|v_k^{(n+1)}|}, \quad \forall k \in \Omega_{\text{PV}} \quad (5)$$

Schritt 3: Q-Update aus der Leistungsbilanz. Der Strom wird aus dem Kirchhoff'schen Knotengesetz bestimmt und die Blindleistung als Imaginärteil der komplexen Leistung berechnet:

$$i_k^{(n+1)} = \sum_{l \in \Omega_d} Y_{kl} \cdot v_l^{(n+1)} + (Y_{ds})_k \cdot v_s \quad (6)$$

$$Q_k^{(n+1)} = \left(v_k^{(n+1)} \cdot \left(i_k^{(n+1)} \right)^* \right), \quad \forall k \in \Omega_{\text{PV}} \quad (7)$$

Schritt 4: Leistungsvektor aktualisieren:

$$s_k^{(n+1)} = P_k + \cdot Q_k^{(n+1)}, \quad \forall k \in \Omega_{\text{PV}} \quad (8)$$

Dieser aktualisierte Leistungsvektor wird im nächsten Iterationsschritt in (4) verwendet.

Wesentliche Eigenschaften. Die Projektion (5) erzwingt die Spannungsbedingung $|v_k| = |V_k|$ in jeder Iteration exakt, nicht nur asymptotisch. Die Blindleistung in (7) ergibt sich direkt aus der Netzgleichung ohne Linearisierung oder Sensitivitätsapproximation.

3.3 Referenzmethoden

Als Vergleichsmethoden dienen:

- Newton-Raphson mit PV-Knoten (Sparse, pandapower)
- Standard-TPF ohne PV [1] (als obere Schranke für den Speedup)

3.4 Bewertungsmetriken

Tabelle 2: Bewertungsmetriken

Metrik	Definition
Spannungsfehler	$\epsilon_V = \max_k V_k - V_k^{\text{NR}} $
Blindleistungsfehler	$\epsilon_Q = \max_k Q_k - Q_k^{\text{NR}} $
Iterationen	Anzahl bis Konvergenz ($\epsilon < 10^{-6}$ p.u.)
Rechenzeit	Wall-Clock-Time für alle τ Lastflüsse
Speedup	$t_{\text{NR}}/t_{\text{Methode}}$
Konvergenzrate	Kontraktionsfaktor η nach Proposition 2 in [2]

3.5 Implementierung

Tabelle 3: Software- und Hardware-Umgebung

Aspekt	Details
Programmiersprache	Python 3.x
CPU-Bibliotheken	NumPy, SciPy, pandapower
GPU-Bibliotheken	CuPy oder PyTorch (optional)
Validierung	pandapower (Newton-Raphson mit PV)
Visualisierung	Matplotlib
Hardware	Standard-Laptop (Intel Core i7, NVIDIA GPU)

4 Vorläufige Gliederung

Tabelle 4: Geplante Gliederung der Bachelorarbeit

Kap.	Inhalt	Seiten (ca.)
1	Einleitung (Motivation, Zielsetzung, Aufbau)	5
2	Grundlagen der Lastflussberechnung	12
2.1	Netzmodell und Knotentypen (Slack, PQ, PV)	
2.2	Newton-Raphson-Verfahren mit PV-Knoten	
2.3	Current Injection Method (CIM) und PV-Behandlung [3]	
2.4	Fixed-Point-Iteration und Konvergenz (Banach) [2]	
3	Der Tensor Power Flow [1]	10
3.1	Grundalgorithmus und Konvergenzbeweis	
3.2	Dense- und Sparse-Formulierung	
3.3	Parallelisierung (CPU/GPU)	
3.4	Limitation: Keine PV-Knoten	
4	Methoden zur PV-Integration im TPF	15
4.1	Methode A: Äußere Q-Schleife	
4.2	Methode B: Eingebettete Q-Korrektur	
4.5	Tensorformulierung der Methoden	
5	Implementierung und Testdesign	5
6	Ergebnisse und Diskussion	12
6.1	Validierung gegen NR-Referenz	
6.2	Konvergenzverhalten (η , λ , n_{PV})	
6.3	Rechenzeitvergleich (Skalierung mit τ und $b\phi$)	
6.4	Vergleich der drei Methoden	
6.5	Vergleich mit konventionellen Verfahren	
7	Zusammenfassung und Ausblick	3
Gesamt (ohne Anhang)		~60

5 Zeitplan (10 Wochen)

Tabelle 5: Detaillierter Zeitplan

Woche	Phase	Tätigkeit	Meilenstein
1	3* Grundlagen	Literaturrecherche; Papers [1], [2], [3], [4] im Detail durcharbeiten; mathematische Grundlagen erarbeiten	Verständnis der Methoden
2		Implementierung: Standard-TPF (Algorithm 1 aus [1]); Validierung gegen pandapower	TPF läuft korrekt
3		Implementierung: NR mit PV als Referenz (pandapower-Wrapper); Testnetze aufsetzen	Referenzlösungen verfügbar
4	2* Impl.	Implementierung: Methode A (äußere Schleife) und Methode B (interleaved); Herleitung $N_k \cdot Z_{B,kk} \cdot L_k$ numerisch verifizieren	Methode A+B implementiert
5		Implementierung: Methode C (quadratische Gleichung); Debugging und Validierung aller Methoden gegen NR-Referenz	Alle Methoden validiert
6	2* Sim.	Simulationskampagne: Variation von n_{PV} , τ , λ , $b\phi$; Datenerfassung (Iterationen, Rechenzeit, Fehler)	Rohdaten vollständig
7		GPU-Tests (CuPy); Auswertung und Visualisierung der Ergebnisse; Speedup-Tabellen erstellen	Ergebnisse ausgewertet
8	3* Schreiben	Schreiben: Kapitel 1–3 (Einleitung, Grundlagen, TPF)	Theorieteil fertig
9		Schreiben: Kapitel 4–6 (Methoden, Implementierung, Ergebnisse)	Hauptteil fertig
10		Schreiben: Kapitel 7, Zusammenfassung, Korrekturlesen, Formatierung, Abgabe	Abgabe

6 Abgrenzung

- Die Arbeit fokussiert auf **Verteilnetze** (radial, ggf. schwach vermascht).
- Es werden **keine** neuen formalen Konvergenzbeweise geführt. Der Banach-Beweis für den PQ-Fall wird aus [1], [2] übernommen; für die PV-Erweiterung wird lediglich numerisch validiert.
- Übertragungsnetze mit vielen PV-Knoten werden nicht betrachtet.
- Q-Limits (PV→PQ-Umwandlung bei Erreichen der Blindleistungsgrenzen) werden als einfache Clipping-Operation implementiert, aber nicht im Detail analysiert.

Literaturverzeichnis

- [1] E. M. Salazar Duque, J. S. Giraldo, P. P. Vergara, P. H. Nguyen und H. Sloomweg, "Tensor Power Flow Formulations for Multidimensional Analyses in Distribution Systems," *arXiv preprint arXiv:2403.04578v1*, 2024.
- [2] J. S. Giraldo, O. D. Montoya, P. P. Vergara und F. Milano, "A fixed-point current injection power flow for electric distribution systems using Laurent series," *Electric Power Systems Research*, Jg. 211, S. 108–126, 2022.
- [3] P. A. N. Garcia, J. L. R. Pereira, S. Carneiro Jr., V. M. da Costa und N. Martins, "Three-Phase Power Flow Calculations Using the Current Injection Method," *IEEE Trans. Power Systems*, Jg. 15, Nr. 2, S. 508–514, Mai 2000.
- [4] E. P. dos Santos, B. Kobay, M. A. da Rosa, R. de Souza Salgado und D. Issicaba, "Modified tensor method to power flow analysis," *IET Generation, Transmission & Distribution*, Jg. 13, Nr. 17, S. 3960–3967, 2019.

7 Benötigte Ressourcen

Tabelle 6: Benötigte Ressourcen

Ressource	Details
Software	Python 3.x, NumPy, SciPy, pandapower, Matplotlib, CuPy (optional)
Hardware	Standard-Laptop mit NVIDIA GPU (für GPU-Tests)
Daten	IEEE-Testnetze (öffentlich verfügbar), synthetische Netze (generiert via k-ary tree [1])
Code-Referenz	GitHub-Repository, TensorPowerFlow [1]

Unterschriften

Studierender

Ort, Datum

Betreuer

Ort, Datum